

Requirements `robot_command`

Version v0.01

Table of Contents

1. Robot Programming Languages (RPLs)	1
1.1. Common Industrial Robot Commands	2
1.1.1. Pose representation	2
1.1.2. MoveL - Move Linear	2
1.1.3. MoveJ - Move Joint	2
1.1.4. MoveC - Move Circular	2
1.1.5. Set Linear/TCP Velocity / Feedrate	3
1.1.6. Set Joint Velocity	3
1.1.7. Set Linear/TCP Acceleration	3
1.1.8. Set Joint Acceleration	3
1.1.9. Set TCP / Tool Pose	3
1.1.10. Set Digital Output	3
1.1.11. Wait for Digital Input	3
1.1.12. Set Blend Radius / Tolerance	3
1.1.13. Pause / Wait / Dwell	3
1.1.14. Gripper commands	3
1.1.15. Additional commands	4
1.1.15.1. Speed / Continuous Move	4
1.1.15.2. Force Mode	4
1.1.15.3. Teach / Freedrive Mode	4
1.1.15.4. User Interaction	4
2. RPL for ROS	4
2.1. MoveIt	4
2.1.1. Resources	4
2.1.2. MoveIt Command Interpreter	4
2.1.3. Pose representation	5
2.1.4. Implementing the commands	5
2.1.4.1. Movement commands	5
2.1.4.2. MoveL	5
2.1.4.3. MoveC	5
2.1.4.4. MoveJ	6
2.1.4.5. Set Linear velocity and acceleration	6
2.1.4.6. Set Joint velocity and acceleration	6
2.1.4.7. Set TCP / Tool Pose	6
2.1.4.8. Set Blend Radius / Tolerance	6
2.1.4.9. Other commands	6
2.1.5. Using the MoveIt Python Interface as RPL	7
2.2. Canonical Robot Command Language (CRCL)	7

2.3. Using GCode as RPL	8
2.4. Using Python as RPL	9
2.4.1. Is the AST compatible between different Python versions?	9
2.4.2. Is the AST compatible between different Python implementations? (CPython, PyPy, IPython, ...)	10
2.4.3. How can we preserve user comments when modifying the AST?	10
2.4.4. How can we preserve/detect generated conversational code?	10
2.4.4.1. wrapping generated code into functions	10
2.4.4.2. use strings to detect generated blocks	10
2.4.4.3. Alternative to using the AST	10
2.5. Creating a new RPL	10
3. Appendix	11
3.1. URScript	11
3.1.1. Move Commands	12
3.1.1.1. MoveJ	12
3.1.1.2. MoveL	12
3.1.1.3. MoveP	12
3.1.1.4. ServoC	12
3.1.1.5. ServoJ	12
3.1.1.6. SpeedJ	12
3.1.1.7. SpeedL	12
3.1.2. Other interesting commands	12

The goal of this document is to outline different concepts for robot programming or robot command languages. Moreover, different approach for implementing the such an RPL are discussed.

1. Robot Programming Languages (RPLs)

RPLs are what GCode is for CNC machines. However, GCode is a lot more standardized than RPLs. Every manufacturer seems to have it's own RPL designed for their specific robot motion control system.

At the time of writing this document, there is no such thing as an open RPL standard.

Taking a look at the RPLs from different manufacturers, we see that some use a GCode-like syntax, others use XML or even simple script languages.

Using a RPL is only one possible approach for creating a robot program. This approach is widely used in industrial robot arms where programs are usually created via sophisticated CAM software or other high-level GUI software that generates the robot program offline.

Other programming methods for robots include:

- **Dialog based programming:** Is mostly used in combination with teaching and a scripting or RPL interface.
- **3D GUI programming:** Presents the operator with graphical 3D representation of the robot and its environment.
- **Colloborative and teaching mode:** Cobots feature a mode where operators can push the robot in place and record it's movement and position. Recent Cobots feature a combination of teaching and dialog based interfaces, often combined with a physical HMI mounted on the robot itself.
- **Other methods:** This includes graphical programming, pendant programming, programming by demonstration, other type of GUIs and fully automated AI.

From the above methods for programming a robot, dialog based programming an colloborative teaching mode look like the most attractive for novice users.

Fully automated programming would be another interesting option. This would imply the robot knows about it's environment and is capable of object recognition. At the time of writing this document, adding this method for programming the robot looks more like step 2 or 3.

So why do we need a RPL at all? Dialog based programming and teaching are sound options for doing 80% of the task. However, the other 20% of tasks done by a robot are where programming with a RPL becomes viable. This may include interfacing with the environment via APIs, executing movements based on mathematical calculations and so on.

Additionally, the RPL can serve as an abstraction layer to the underlying interpreter or trajectory planner provided by ROS. This gives us additional flexibility in case we want to swtich in the future.

1.1. Common Industrial Robot Commands

Most industrial robots support a very similar set of commands. Looking at postprocessor codes from different CAD/CAM packages, we can see the syntax of the RPLs is very different, but the core commands are very similar.

- [RoboDK Postprocessor scripts](#)
- [RoboDK API](#)

1.1.1. Pose representation

Pose = position of the active tool with respect to the active reference frame

- RoboDK: 4x4 Matrix
- KUKA: XYZABC Euler angles
- ABB: quaternion representation
- UR: XYZUVW position with rotation vector
- Staubli, Fanuc, Motoman: XYZWPR
- Nachi: XYZRPW
- Other: TxyzRxyz target as $[x,y,z,r_x,r_y,r_z]$ so that $H = \text{transl}(x,y,z) * \text{rotx}(r_x) * \text{roty}(r_y) * \text{rotz}(r_z)$

1.1.2. MoveL - Move Linear

Linear movement command. Moves the robot tool head in a straight line from waypoint 1 to waypoint 2.

Command can be invalid if tool would move through the robot arm.

This command is similar to G1 in GCode.

Arguments

- target: target pose or joint positions

1.1.3. MoveJ - Move Joint

Move to position linear in joint space.

Arguments

- target: target pose or joint positions

1.1.4. MoveC - Move Circular

Blend circular and move linear (both in tool-space) to position.

This command is similar to G2 in GCode.

Arguments

- target1: move through this waypoint, pose or joint positions
- target2: final circle target, pose or joint positions

1.1.5. Set Linear/TCP Velocity / Feedrate

Sets the linear speed of a robot

1.1.6. Set Joint Velocity

Sets the joint speed of a robot in deg/s for rotary joints and mm/s for linear joints.

1.1.7. Set Linear/TCP Acceleration

Sets the linear acceleration of a robot in mm/s².

1.1.8. Set Joint Acceleration

Sets the joint acceleration of a robot in deg/s² for rotary joints and mm/s² for linear joints.

1.1.9. Set TCP / Tool Pose

Set the robot tool pose (TCP) with respect to the robot flange.

1.1.10. Set Digital Output

Sets a digital output.

1.1.11. Wait for Digital Input

Wait for a digital input to change / become true.

1.1.12. Set Blend Radius / Tolerance

Set the path blending tolerance for joint or linear movement commands.

This command is similar to G64 in GCode.

1.1.13. Pause / Wait / Dwell

Wait for n seconds/milliseconds.

1.1.14. Gripper commands

The gripper open/close commands seems to depend on the type of gripper. This might be for example a set digital out command combined with a wait statement.

1.1.15. Additional commands

1.1.15.1. Speed / Continuous Move

Move the robot joints or TCP with a certain speed.

1.1.15.2. Force Mode

Move the robot in force mode.

1.1.15.3. Teach / Freedrive Mode

Enables or disabled the teaching or freedrive mode, where the motors are driven in a way that they just overcome gravity.

1.1.15.4. User Interaction

Shows a message box or dialog in the UI. Waiting for user input to continue the process or to enter a value.

2. RPL for ROS

ROS doesn't ship with any standard RPL. The closest that can be found is MoveIt commander or [CRCL](#).

In this section, we take a closer look at different approaches to implement a RPL for our robot.

2.1. MoveIt

This section evaluates the use of MoveIt for execution of commands supported by other RPLs.

The MoveIt motion planner provides with Move Group and MoveIt Commander an interface that could be used as potential backend for an RPL interpreter.

The core idea is to feed the movement commands from an RPL interpreter (which could be Python) directly into MoveIt to generate trajectories.

2.1.1. Resources

- [Move Group Interface](#)
- [Move Group Command API](#)
- [Matilda control](#)

2.1.2. MoveIt Command Interpreter

MoveIt commander has a [simple interpreter interface](#).

```
roslaunch moveit_commander moveit_commander_cmdline.py
```

2.1.3. Pose representation

MoveIt pose is a list of 6 floats: [x, y, z, rot_x, rot_y, rot_z] or a list of 7 floats [x, y, z, qx, qy, qz, qw], alternatively one can use `geometry_msgs.msg.Pose`.

2.1.4. Implementing the commands

2.1.4.1. Movement commands

MoveIt converts the movement commands to joint trajectories. To get a continuous smooth movement we may need to combine multiple trajectories into one. This can be achieved as explained in the forums: Stitching of trajectories: <https://groups.google.com/forum/#!topic/moveit-users/lZL2HTjLu-k>

2.1.4.2. MoveL

A single movement can be executed with `set_pose_target`, multiple movements can be chained to `compute_cartesian_path`.

```
# first orient gripper and move forward (+x)
wpose = deepcopy(target)
print(wpose)
#wpose.orientation.w = 1.0
wpose.position.x = 0.3
waypoints.append(deepcopy(wpose))

# second move down
wpose.position.z = 1.10
waypoints.append(deepcopy(wpose))

# third move to the side
wpose.position.y = 0.3
waypoints.append(deepcopy(wpose))

(plan3, fraction) = manipulator.compute_cartesian_path(
    waypoints,   # waypoints to follow
    0.01,       # eef_step
    0.0)        # jump_threshold
manipulator.execute(plan3)
```

2.1.4.3. MoveC

MoveIt does not support circular movements out of the box. However, we can construct a circular movement from a list of cartesian waypoints. So the executions should be similar to MoveL.

2.1.4.4. MoveJ

Can be implemented with `set_joint_value_target`:

```
joints = manipulator.get_current_joint_values()
joints[0] = 0.5
manipulator.set_joint_value_target(joints)
manipulator.go()
```

2.1.4.5. Set Linear velocity and acceleration

Setting the linear velocity (end effector velocity) for motion planning is not supported by MoveIt.

However, using the methods in `moveit_core/trajectory_processing` one could post process the planned trajectory to enable constant TCP velocity. (<https://answers.ros.org/question/272459/how-to-control-cartesian-velocity/>)

2.1.4.6. Set Joint velocity and acceleration

Setting the joint velocity can be achieved by setting the `set_max_velocity_scaling_factor`.

Setting the joint acceleration can be achieved by setting the `set_max_acceleration_scaling_factor`.

2.1.4.7. Set TCP / Tool Pose

MoveIt is aware of the attached tool or object and, therefore, setting the TCP is equivalent to changing the end effector or to attach a different object.

An alternative would be to introduce an internal TCP / pose value added to all poses send to moveit.

2.1.4.8. Set Blend Radius / Tolerance

MoveIt provides multiple tolerance parameters:

`set_goal_joint_tolerance`, `set_goal_orientation_tolerance`, `set_goal_position_tolerance` and the arguments of `compute_cartesian_path`.

2.1.4.9. Other commands

Other commands such as for examples set digital output are very likely not executed via the MoveIt interface.

- Force Mode: <http://www.ros.org/news/2016/09/ros-interface-for-impedanceforce-control.html>
- Continuous Mode
- Teach Mode
- User Interaction

2.1.5. Using the MoveIt Python Interface as RPL

As we see, MoveIt commander already supports most of the movement commands out of the box. Therefore, using MoveIt commander as possible RPL might be an option. Commands not support by MoveIt commander could be implemented and added to the API.

To simplify writing programs for the user, one could either create boilerplate templates or move the boilerplate code into a support module.

However, when we use MoveIt commander as is, we don't have access to the context of the currently executed command/code such as the line number. It might be possible to achieve this by using gdb/pdb, but I don't consider this a viable option.

Rereading of generated Python code is discussed in the chapter dedicated to using Python as RCL.

2.2. Canonical Robot Command Language (CRCL)

CRCL is a NIST standard for robot control with an existing [ROS implementation](#).

CRCL is intended to be used as common standard to interface with industrial robots.

Program instances are defined in XLM format assisted by XML schema files. Here is an example a move command:

```
<?xml version="1.0" encoding="UTF-8"?>
<CRCLCommandInstance
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="../xmlSchemas/CRCLCommandInstance.xsd">
  <CRCLCommand xsi:type="MoveToType">
    <CommandID>2</CommandID>
    <MoveStraight>>false</MoveStraight>
    <EndPosition>
      <Point>
        <X>2.5</X> <Y>1</Y> <Z>1</Z>
      </Point>
      <XAxis>
        <I>1</I> <J>0</J> <K>0</K>
      </XAxis>
      <ZAxis>
        <I>0</I> <J>0</J> <K>-1</K>
      </ZAxis>
    </EndPosition>
  </CRCLCommand>
</CRCLCommandInstance>
```

CRCL has a set of pre-defined commands which cover the most used operations of an industrial robot. This includes for example moves in cartesian or joint space and open/close of the gripper.

CRCL is designed to work for both open-loop and closed-loop control.

The primary application for CRCL is one-off (lot of 1) manufacturing, where programs are generated by a computer. To assist this application, the CRCL also make use of the robot model. Moreover, the standard is accompanied by open standards for robot ontology.

Advantages of using CRCL include:

- using a existing standard
- designed with robot/factory ontology in mind
- XML can be auto-generated easily
- reference implementations for Python and ROS exist

Disadvantages of using CRCL:

- XML format is not human friendly
- I haven't seen any control structures
- adding new commands means editing the schema
- adoption by other vendors is unclear

2.3. Using GCode as RPL

GCode is designed for linear movements in cartesian space. Robots, however, are controlled either in cartesian space moving the tool head around or by controlling the joints directly. Direct joint movement is not supported by standard RS-247 GCode.

This is also a problem when working with for example Scara or Delta 3D printers. Therefore, the regrab community added some new GCode commands such as [G6](#) to drive motors/joint directly. However, the commands are usually designed for calibration purposes and not for normal operation.

Although GCode has control structures such as loops when combined with O-Codes, they are usually cumbersome to work with when writing the program by hand. Further limitations include variables and scoping, rather hard extension of the interpreter and so on.

One advantage of using GCode as RPL is that CNC machine operators are already familiar with how to work the GCode programs.

Deciding to use GCode would practically mean writing a new GCode interpreter or reusing an existing one and extending it with joint movement commands.

Given that GCode has it's limitations and there is no such thing as a common RPL standard based on GCode at the moment, I don't think that's the best way to go forward.

Additional findings:

- [B&R robot controllers use GCode.](#)
- [Sinumerik may also be used to control a robot.](#)

2.4. Using Python as RPL

Python is a very powerful and easy to use programming language. Using Python as programming interface for a robot by exposing the robot functions as Python module makes a lot of sense. Since Python is a very popular programming language, users can easily learn the basics of the language from different sources and tutorials.

One can think of either executing the commands synchronously or asynchronously. Executing the commands synchronously would be easier to implement and probably also more intuitive.

However, considering CAM generated programs as a possible input source for the robot, executing commands synchronously in sequence wouldn't result in a smooth movement, which is a required for manufacturing applications.

A possible alternative is to create asynchronous commands, batching together multiple waypoints for movement commands. When hitting a command that breaks the motion queue, the previous command is dispatched as whole and motion planning or playout can be executed in a smooth manner.

The following things need to be considered:

- What is the maximum number of movements that can be batched together?
- What happens when this number is exceeded (do we need a motion queue)?
- Which commands are "queue breakers".
- How can we deal with custom code and syntax/code errors? (linter?)
- How can we implement more complex elements such as threads?
- How can we re-read a program for the dialog based interface?

The last point might be hard to tackle. Since control constructs such as a loop can't be easily dissected by executing the code in "read mode". Of course we could just read the source and try to figure out the command syntax and parameters, but that seems rather hard for a complete programming language such as Python.

Another option would be to use a similar approach to job assignments in PathPilot. This would mean appending the job settings as source comments or code. User modifications would then be lost when the source code is re-read.

After some more research I found out that we can use the [Python AST](#) to dissect the Python program code. It even seems possible to modify the Python grammar if necessary.

Considerations when using the AST:

2.4.1. Is the AST compatible between different Python versions?

The AST itself is not compatible between different Python versions. However, the `ast` module is part of the Python standard library and considered stable.

2.4.2. Is the AST compatible between different Python implementations? (CPython, PyPy, IPython, ...)

The AST itself probably not, but the `ast` module is available in all tested Python implementations.

2.4.3. How can we preserve user comments when modifying the AST?

Comments are not part of the Python AST. As alternative we can use the `tokenize` module, which is part of the Python standard library. Examples for combining `ast` and `tokenize` exist.

2.4.4. How can we preserve/detect generated conversational code?

Preserving and reading the generated code can be done by analyzing the AST of the generated code blocks.

Options to detect generated code include:

2.4.4.1. wrapping generated code into functions

```
def linear_move_1():
    ''' linear move '''
    move1(p[0, 0, 0, 0, 0, 0])
    move1(p[0, 2.0, 0, 0, 0, 0])
    move1(p[10, 0, 0, 0, 0, 0])
linear_move_1()
```

2.4.4.2. use strings to detect generated blocks

```
" linear move 1 "
move1(p[0, 0, 0, 0, 0, 0])
move1(p[0, 2.0, 0, 0, 0, 0])
move1(p[10, 0, 0, 0, 0, 0])
```

Note that strings can be found in the AST, comments are excluded.

2.4.4.3. Alternative to using the AST

An alternative for reading the program source code is to use the Python `tokenize` module and to ignore the syntax tree. This wouldn't allow us to analyze the program structure. However, it makes reading the program probably far simpler, especially considering more complex programs.

To detect generated code blocks we could use the methods mentioned above. Reading of parameters could be done by analyzing the AST or by including the parameters in the comments.

2.5. Creating a new RPL

Creating a new custom RPL has some advantages over using an existing scripting language such as

Python. Most importantly we can create our own DSL specially targeted for robots. In Python, some construct that are useful for a robot might be more complex to implement. This includes threading for example.

Moreover, we don't have control over the Python interpreter itself. Running the code synchronously would very likely lead to performance problems with CAM generated "naive" code. Additionally, more complex constructs such as loops or threads might be hard to emulate using pure Python.

Therefore, creating a custom DSL with one of the parser generator languages supported by Python is a viable option.

This page gives an overview of parser generators for Python: <https://wiki.python.org/moin/LanguageParsing>

I took a closer look at yapps, because it's also used in Machinekit and [textX](#). Whereas textX seems to be easier to use.

Especially interesting in this context is the [robot DSL tutorial](#) provided in the textX documentation.

The [Deborah Script example](#) demonstrates how to use textX to create an interpreter in combination with Python's ast module.

Although creating a new DSL seems attractive at a first glance, maintaining and implementing a complete interpreter would still require a lot of effort. Considering that the 80/20 rule applies to robot programming, whereas users program the 80% of the time using dialog or teaching interfaces, the effort required to build a new RPL seems to be too high.

In my opinion, creating a new robot DSL is an option, however, only if Python as RPL doesn't satisfy our requirements. Development and long time maintenance of a new DSL would require a lot of effort, which is not justifiable at the moment.

3. Appendix

3.1. URScript

Let's take a closer look at URScript.

URScript is pretty interesting because it looks very similar to Python. Moreover, the UR GUI called PolyScope generates URScript code in the background - and is also able to re-read code blocks after editing by the user.

URScript is often referenced as Python. However, when we take a closer look at the language we see that it is a custom DSL with a syntax similar to Python.

API reference: http://www.sysaxes.com/manuels/scriptmanual_en_3.1.pdf

Most interesting commands:

3.1.1. Move Commands

3.1.1.1. MoveJ

Move to position linear in joint space.

3.1.1.2. MoveL

Linear movement command. Moves the robot tool head in a straight line from waypoint 1 to waypoint 2.

Command can be invalid if tool would move through the robot arm.

3.1.1.3. MoveP

Blend circular and move linear (both in tool-space) to position.

3.1.1.4. ServoC

Servo to position (circular in tool-space). Accelerates to and moves with constant tool speed v .

3.1.1.5. ServoJ

Servo to position (linear in joint-space). Used for online control.

3.1.1.6. SpeedJ

Accelerate to and move with constant joint speed.

3.1.1.7. SpeedL

Accelerate to and move with constant tool speed.

3.1.2. Other interesting commands

- `set_tcp`
- `set_gravity`
- `set_payload`
- `force_mode`